Module 2: Variables and Builtin Variables

User-Defined Variables

Variables allow you to define values once and reuse them throughout your playbook. This makes playbooks portable: change the target IP in one place and every command updates automatically.

```
vars:
  METASPLOITABLE: 172.17.0.106
  PASSWDLIST: /usr/share/seclists/Passwords/darkweb2017-top1000.txt

commands:
  - type: shell
    cmd: nmap -A -T4 $METASPLOITABLE

  - type: shell
    cmd: hydra -l user -P $PASSWDLIST $METASPLOITABLE ftp
```

Key Rules

1. **Definition**: No `$` required (but allowed) in the `vars` section
2. **Reference**: `$` prefix is **required** in the `commands` section
3. **Storage**: All values are stored as strings internally, even numbers
4. **Substitution**: Uses Python's `string.Template` syntax

Environment Variable Override

Any playbook variable can be overridden by setting an environment variable with the `ATTACKMATE_` prefix:

```
# Override the METASPLOITABLE variable without editing the playbook
export ATTACKMATE_METASPLOITABLE=10.10.10.50
attackmate playbook.yml
```

This is useful for running the same playbook against different targets without modification.

Builtin Variables

AttackMate automatically sets several variables during execution:

Variable	Description
`$RESULT_STDOUT`	Standard output of the most recently executed command
`$RESULT_RETURNCODE`	Return code of the most recently executed command

Variable	Description
$LAST_MSF_SESSION	Session number when a Metasploit session is created
$LAST_SLIVER_IMPLANT	Path to a newly generated Sliver implant
$REGEX_MATCHES_LIST	List of all matches from the last regex command

The two most important ones for Day 1 are $RESULT_STDOUT and $RESULT_RETURNCODE.

RESULT_STDOUT

After every command execution, the output is stored in $RESULT_STDOUT. This is how you chain commands together, one command produces output, the next command consumes it.

```
commands:
  # Step 1: Run nmap (its output is stored in $RESULT_STDOUT)
  - type: shell
    cmd: nmap -p 80 192.168.1.100

  # Step 2: The regex command reads $RESULT_STDOUT by default
  - type: regex
    cmd: (\d+)/tcp open  http
    output:
      PORT: "$MATCH_0"

  # Step 3: Use the extracted value
  - type: debug
    cmd: "HTTP port found: $PORT"
```

> **Note**: debug, regex, and setvar commands do **not** overwrite $RESULT_STDOUT. This means you can run multiple regex/debug commands after a shell command without losing the original output.

RESULT_RETURNCODE

The return code lets you check whether a command succeeded (0) or failed (non-zero):

```
commands:
  - type: shell
    cmd: ping -c 1 192.168.1.100

  - type: debug
    cmd: "Ping returned: $RESULT_RETURNCODE"
```

Setting Variables Dynamically

The setvar Command

Use `setvar` to create or modify variables during playbook execution:

```
commands:
  - type: setvar
    variable: GREETING
    cmd: Hello World

  - type: debug
    cmd: $GREETING
```

The `variable` field specifies the name (without `$`), and `cmd` provides the value. Variable substitution works in `cmd`, so you can combine existing variables:

```
vars:
  FIRST: Hello
  SECOND: World

commands:
  - type: setvar
    variable: COMBINED
    cmd: "$FIRST $SECOND"

  - type: debug
    cmd: $COMBINED
```

Encoding Support

`setvar` has a built-in `encoder` option for common transformations:

```
commands:
  - type: setvar
    variable: ENCODED
    cmd: Hello World
    encoder: base64-encoder

  - type: debug
    cmd: $ENCODED
    # Output: SGVsbG8gV29ybGQ=
```

Available encoders: `base64-encoder`, `base64-decoder`, `rot13`, `urlencoder`, `urldecoder`

List Variables

Variables can also hold lists of values. List elements are accessed by index:

```
vars:
  TARGETS:
    - 192.168.1.100
    - 192.168.1.101
    - 192.168.1.102

commands:
  - type: debug
    cmd: "First target: $TARGETS[0]"

  - type: debug
    cmd: "Second target: $TARGETS[1]"
```

Lists become particularly powerful when combined with the `loop` command (covered in Day1 Module 4).

The `debug` Command

The `debug` command is your primary tool for inspecting variables and troubleshooting playbooks:

```
commands:
  - type: debug
    cmd: "Current target: $TARGET"
```

Dump All Variables

Use `varstore: True` to print every variable currently stored:

```
commands:
  - type: shell
    cmd: echo "test output"

  - type: debug
    varstore: True
```

Pause Execution

Use `wait_for_key: True` to pause and wait for the user to press Enter. Useful for stepping through a playbook manually:

```
commands:
  - type: shell
    cmd: nmap $TARGET

  - type: debug
    cmd: "Scan complete. Review the output above."
```

```
    wait_for_key: True

  - type: shell
    cmd: nikto -host $TARGET
```

Early Exit

Use `exit: True` to stop the playbook at a specific point:

```
commands:
  - type: shell
    cmd: nmap $TARGET

  - type: debug
    cmd: "Stopping here for debugging"
    exit: True

  # These commands will NOT execute:
  - type: shell
    cmd: nikto -host $TARGET
```

> **Tip**: Run AttackMate with `--debug` for higher verbosity. It qill print variable dumps after regex commands and detailed execution information for every step.